

Authentication Module Documentation

1. Feature NameAuthentication Module

2. Feature Overview

The Authentication module is responsible for verifying the identity of users before allowing access to the Enterprise Workforce Management Platform. It ensures that only registered and authorized users can log in to the system using their Employee ID (or Email) and Password.

The module also manages secure user sessions using JSON Web Tokens (JWT), password validation, account security, and role-based access to different dashboards.

3. Objectives

- Authenticate registered employees securely.
 - Prevent unauthorized access.
 - Generate secure login sessions using JWT.
 - Redirect users based on their assigned role.
 - Protect sensitive application resources.
 - Improve system security through password encryption and account lock mechanisms.
-

4. Components of Authentication

Frontend Components

- Login Page (Login.jsx)
 - Login Form
 - Form Validation
 - API Service (backendAPI.js)
 - Protected Routes
-

Backend Components

- Authentication API
 - Login Controller
 - Authentication Middleware (MiddlewareAuth.js)
 - JWT Token Generator
 - Password Verification
 - Role Verification
-

Database Components

- Employee Table
 - User Credentials
 - Role Information
 - Login Attempt Counter
 - Account Status
-

5. Authentication Workflow

Step 1 – User Opens Login Page

The employee opens the Login page and enters:

- Employee ID / Email
 - Password
-

Step 2 – Frontend Validation

Before sending the request, the frontend checks:

- Employee ID is not empty
- Password is not empty
- Input format is valid

If validation fails, the user receives an error message.

Step 3 – API Request

The frontend sends the login request through:

```
backendAPI.js
```

Example API Request

```
POST /api/auth/login
```

Request Body

```
{  
  "employeeId": "EMP001",
```

```
"password": "*****"  
}
```

Step 4 – Authentication Middleware

The request reaches the backend.

```
Login Route  
  
↓  
  
MiddlewareAuth.js
```

The middleware performs:

- Request validation
- Input sanitization
- Security checks

Step 5 – Database Verification

The backend queries the Employee database.

```
SELECT * FROM Employees  
WHERE EmployeeID = 'EMP001'
```

The system checks:

- Employee exists
- Account is active
- Account is not locked

Step 6 – Password Verification

The stored encrypted password is compared with the entered password using a hashing algorithm (e.g., bcrypt).

If passwords match:

- Authentication succeeds.

Otherwise:

- Authentication fails.
-

Step 7 – JWT Token Generation

After successful authentication:

- JWT Token is generated.
- User Role is retrieved.
- Session information is created.

Example JWT Payload

```
{
  "employeeId": "EMP001",
  "role": "Employee",
  "exp": "Expiration Time"
}
```

Step 8 – Response to Frontend

The backend returns:

```
{
  "success": true,
  "token": "JWT_TOKEN",
  "role": "Employee"
}
```

Step 9 – Store Session

Frontend stores:

- JWT Token
- User Information

The token is used for authenticated requests.

Step 10 – Redirect User

Based on the user's role:

Role	Redirect To
Admin	Admin Dashboard
HR	HR Dashboard
Manager	Manager Dashboard
Employee	Employee Dashboard
IT Admin	IT Dashboard

6. Authentication Flow Diagram

```
graph TD; A[Employee] --> B[Open Login Page]; B --> C[Enter Employee ID & Password]; C --> D[Frontend Validation]; D --> E[backendAPI.js];
```

The diagram illustrates the authentication flow for an employee. It begins with the 'Employee' entity, which leads to the 'Open Login Page' step. From there, the user proceeds to 'Enter Employee ID & Password', followed by 'Frontend Validation'. The final step in the flow is 'backendAPI.js', which handles the backend logic for authentication.

POST /api/auth/login

↓

MiddlewareAuth.js

↓

Validate Request

↓

Database

↓

Check Employee ID

↓

Employee Found?

├ No

|

├ Return Error

|

└ "Employee does not exist."

↓

Yes

↓

Verify Password

↓

Password Correct?

├ No

|

├ Increase Failed Login Count

|

└ Return Invalid Password

↓

Yes

↓

Generate JWT Token

↓

Create User Session

↓

Return Login Success

↓

Frontend Stores JWT

↓

Redirect to Dashboard

7. Authentication Success Flow

Login Request

↓

Employee Verified

↓

Password Verified

↓

JWT Generated

↓

Session Created

↓

Role Identified

↓

Dashboard Loaded

8. Authentication Failure Flow

Case 1: Employee Not Found

```
Employee Login
↓
Database Search
↓
Employee Not Found
↓
Return Error
↓
Display
"Employee does not exist."
```

Case 2: Incorrect Password

```
Employee Login
↓
Password Verification
↓
Password Incorrect
↓
Increase Failed Login Count
↓
```

Display

"Invalid Password."

Case 3: Account Locked

3 Failed Login Attempts

↓

Account Status Updated

↓

Account Locked

↓

Display

"Your account has been locked. Please contact the IT Administrator."

9. Security Features

- Passwords are stored in encrypted (hashed) form.
 - JWT tokens are used for secure session management.
 - Protected APIs require valid JWT tokens.
 - Role-based access control (RBAC) restricts access to authorized resources.
 - Input validation prevents malformed requests.
 - Failed login attempts are tracked.
 - Accounts are temporarily locked after multiple failed login attempts.
 - Sessions expire automatically after the JWT token expires.
-

10. Inputs and Outputs

Input

- Employee ID / Email
- Password

Output (Success)

- JWT Token
- User Role
- Redirect to appropriate dashboard

Output (Failure)

- Employee does not exist
 - Invalid password
 - Account locked
 - Unauthorized access
-

11. Expected Outcome

After successful authentication, the employee gains secure access to the platform. A JWT-based session is established, the user's role is verified, and they are redirected to the appropriate dashboard. Invalid credentials, inactive accounts, or unauthorized access attempts are blocked, ensuring that only authenticated users can access protected features of the Enterprise Workforce Management Platform.